

Библиотека за визуализиране на данни Matplotlib.

доц. д-р Гергана Спасова
Кат. „Компютърни науки и технологии“

Инсталиране на matplotlib

`pip install matplotlib`

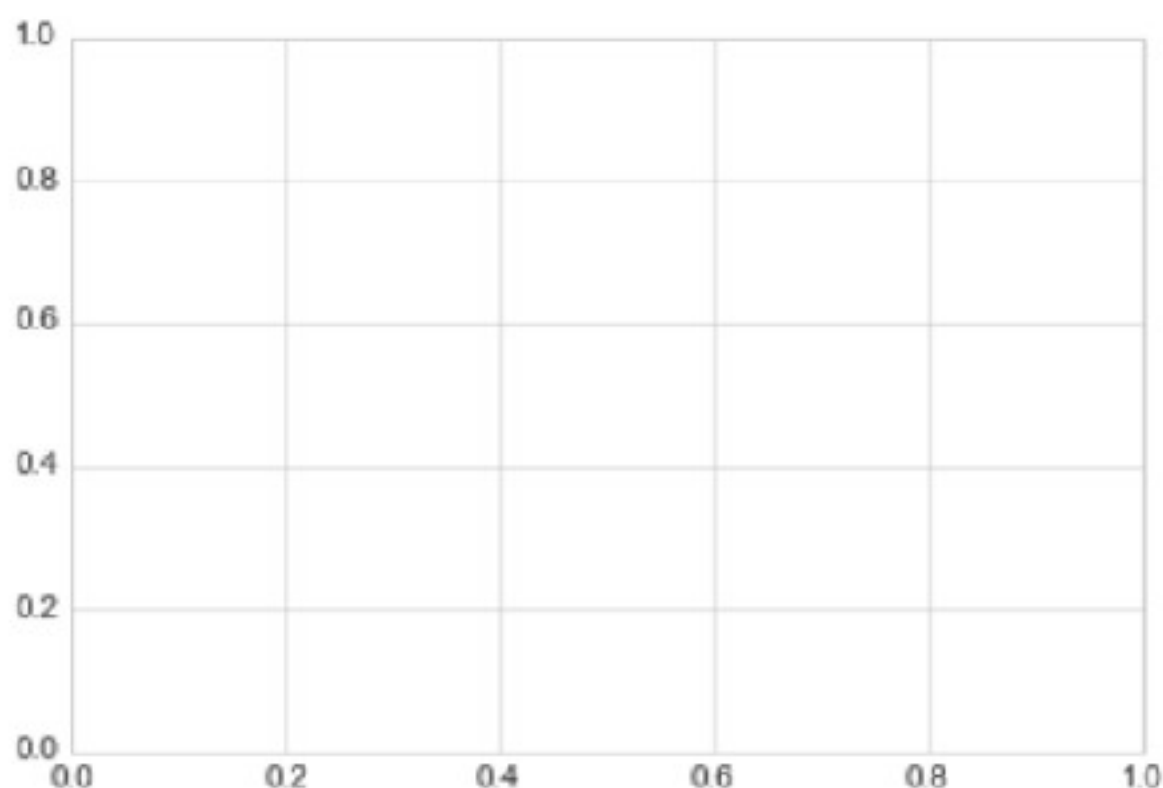
Настройки за начертаване и импортиране на пакети,
които ще се използват:

```
In [1]: %matplotlib inline
import matplotlib.pyplot as plt
plt.style.use('seaborn-whitegrid')
import numpy as np
```

Построяване на линейни диаграми.

В най-простата си форма фигура и оси могат да бъдат създадени по следния начин:

```
In [2]: fig = plt.figure()  
        ax = plt.axes()
```



Построяване на линейни диаграми.

В Matplotlib фигурата може да се разглежда като единичен контейнер, който съдържа всички обекти, представляващи оси, графики, текст и етикети.

След създаването на оси, може да се използва функцията **ax.plot**, за да се изчертаят данни.

Пример: Изчертаване на проста синусоида:

```
In [3]: fig = plt.figure()
ax = plt.axes()

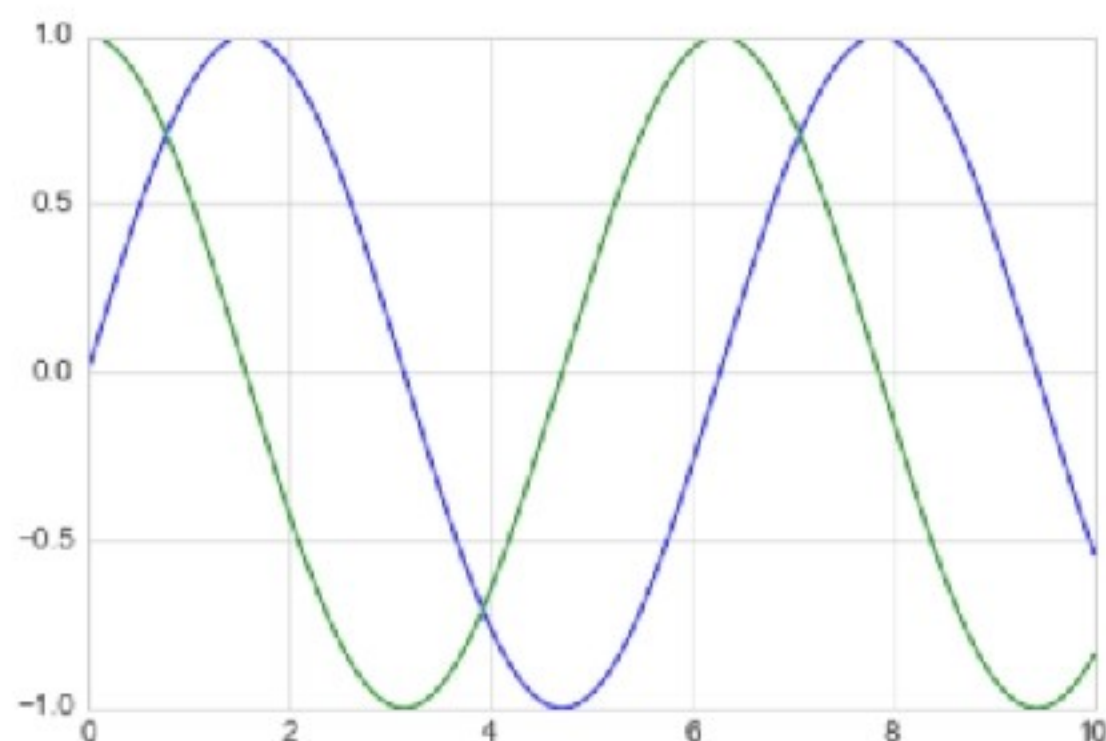
x = np.linspace(0, 10, 1000)
ax.plot(x, np.sin(x));
```



Построяване на линейни диаграми.

При създаване на една фигура с множество линии, може да се извика функцията за графика няколко пъти:

```
In [5]: plt.plot(x, np.sin(x))  
plt.plot(x, np.cos(x));
```



Цветовете и стиловете на линията

Функцията **`plt.plot()`** има допълнителни аргументи, които могат да се използват за задаване на цветовете и стиловете на линията. За настройка на цвета, може да се използва ключовата дума `color`, която приема аргумент за низ, представляващ практически всеки възможен цвят. Цветът може да бъде определен по различни начини.

```
In [6]: plt.plot(x, np.sin(x - 0), color='blue')           # specify color by name
plt.plot(x, np.sin(x - 1), color='g')                   # short color code (rgbcmk)
plt.plot(x, np.sin(x - 2), color='0.75')                # Grayscale between 0 and 1
plt.plot(x, np.sin(x - 3), color='#FFDD44')             # Hex code (RRGGBB from 00 to
plt.plot(x, np.sin(x - 4), color=(1.0,0.2,0.3))          # RGB tuple, values 0 to 1
plt.plot(x, np.sin(x - 5), color='chartreuse');          # all HTML color names support
```



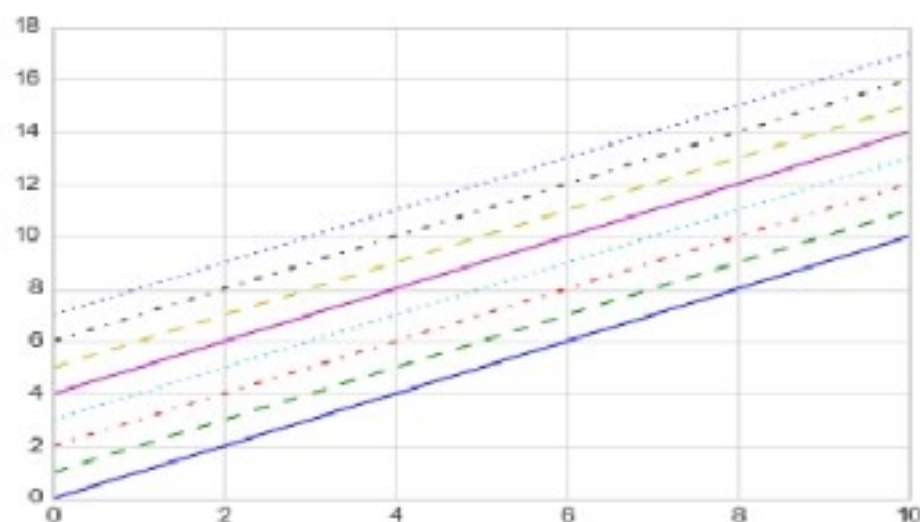
Цветовете и стиловете на линията

Ако не е посочен цвят, Matplotlib автоматично ще премине през набор от цветове по подразбиране за множество линии.

По същия начин стилът на линията може да се коригира с помощта на ключовата дума ***linestyle***:

```
In [7]: plt.plot(x, x + 0, linestyle='solid')
plt.plot(x, x + 1, linestyle='dashed')
plt.plot(x, x + 2, linestyle='dashdot')
plt.plot(x, x + 3, linestyle='dotted');

# For short, you can use the following codes:
plt.plot(x, x + 4, linestyle='-') # solid
plt.plot(x, x + 5, linestyle='--') # dashed
plt.plot(x, x + 6, linestyle='-.') # dashdot
plt.plot(x, x + 7, linestyle=':'); # dotted
```

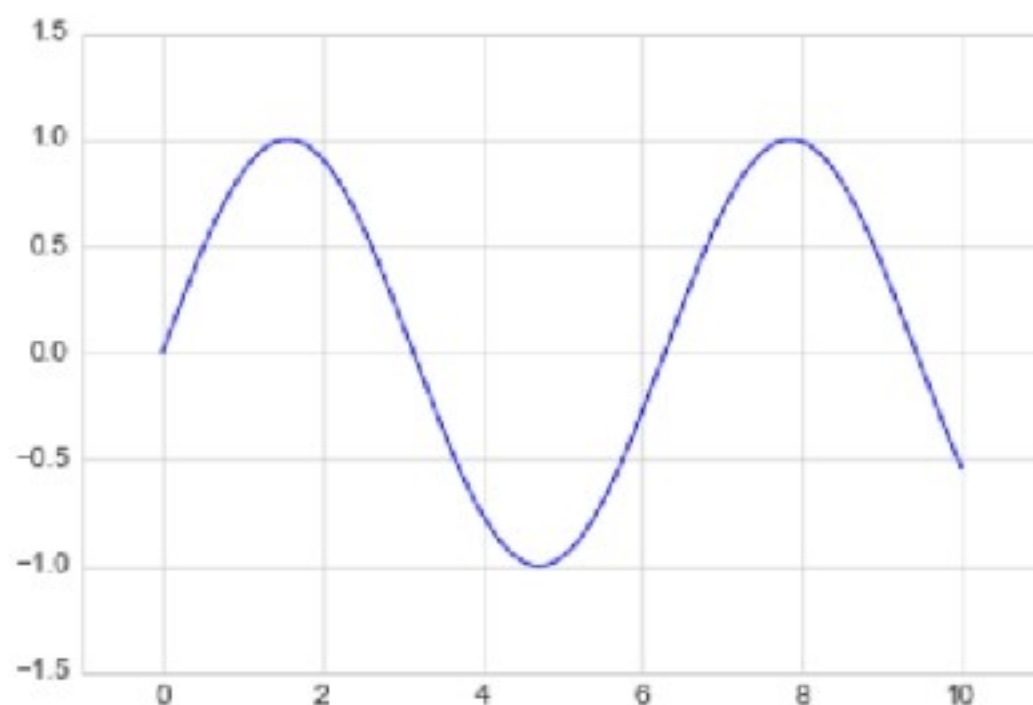


Регулиране на границите на осите

Коригиране на границите на оста може да се направи с използването на методите ***plt.xlim()*** и ***plt.ylim()***:

```
In [9]: plt.plot(x, np.sin(x))
```

```
plt.xlim(-1, 11)  
plt.ylim(-1.5, 1.5);
```



Регулиране на границите на осите

Ако искате да се покаже обратната ос, може да се обърне реда на аргументите:

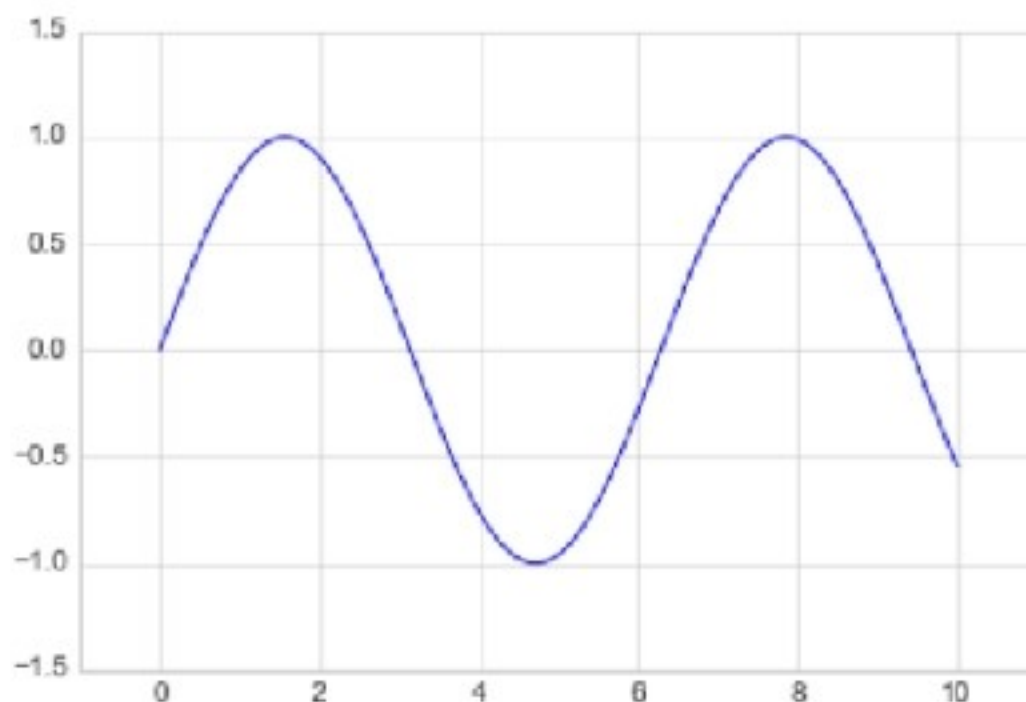
```
In [10]: plt.plot(x, np.sin(x))  
  
plt.xlim(10, 0)  
plt.ylim(1.2, -1.2);
```



Регулиране на границите на осите

Методът **`plt.axis()`** позволява задаване на границите x и y с едно извикване, като се предаде списък, който определя `[xmin, xmax, ymin, ymax]`:

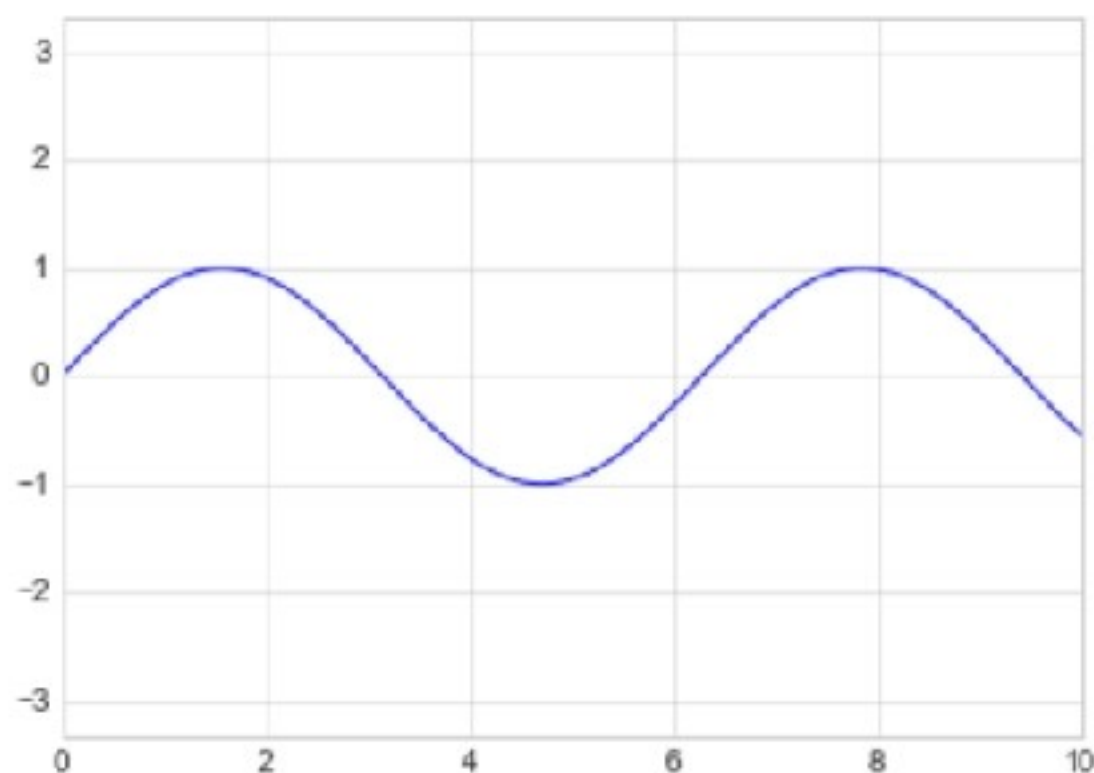
```
In [11]: plt.plot(x, np.sin(x))  
plt.axis([-1, 11, -1.5, 1.5]);
```



Регулиране на границите на осите

Позволява дори спецификации на по-високо ниво, като например осигуряване на еднакво съотношение на страните, така че една единица в x да е равна на една единица в y :

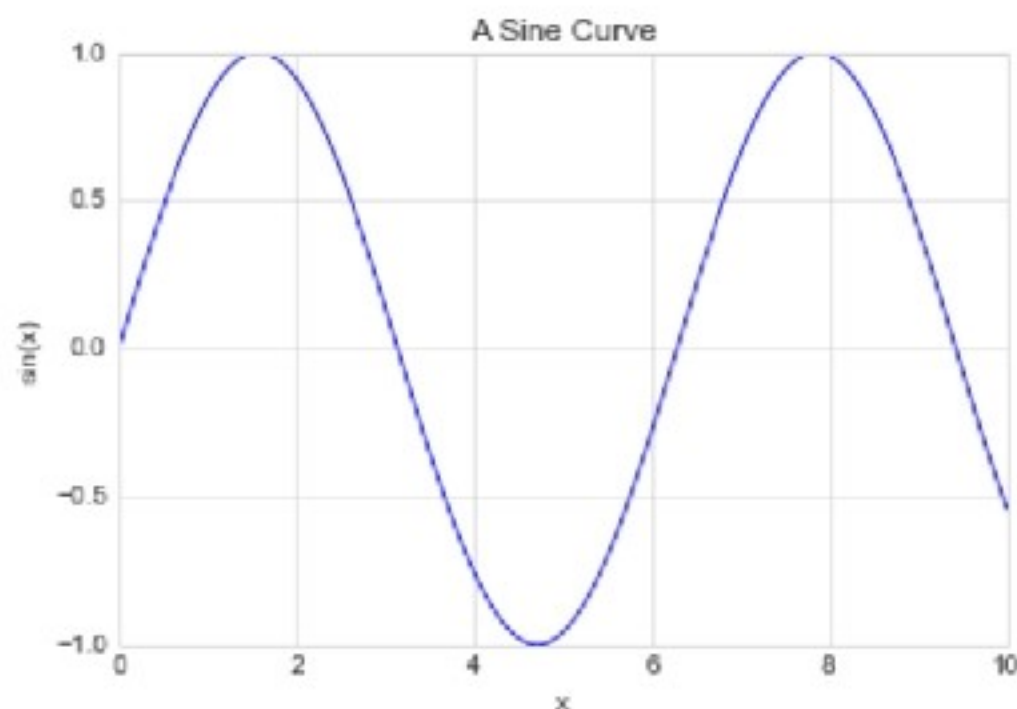
```
In [13]: plt.plot(x, np.sin(x))  
plt.axis('equal');
```



Озаглавяване на оси

Методи за задаване на заглавията и етикетите на оста:

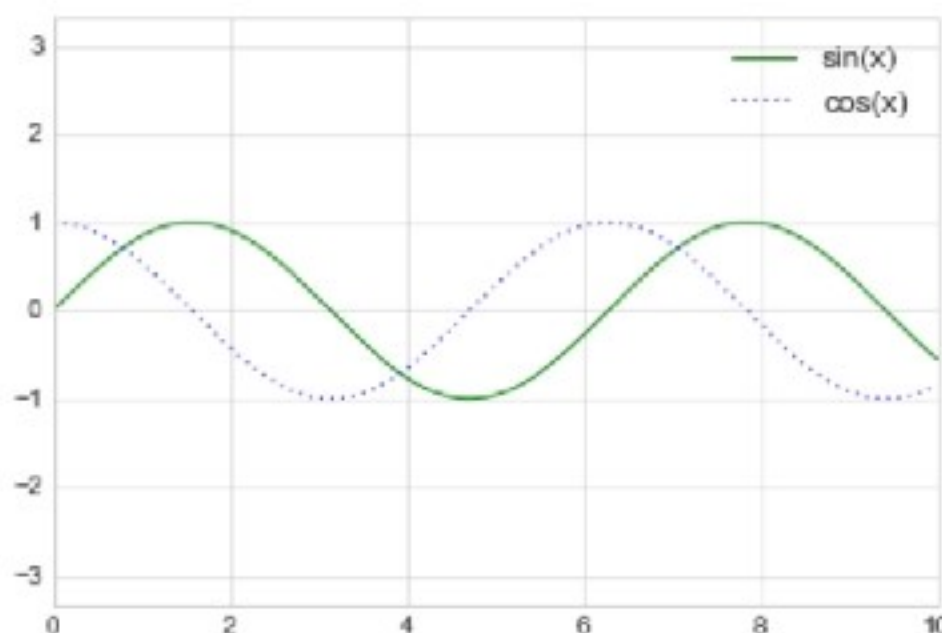
```
In [14]: plt.plot(x, np.sin(x))  
plt.title("A Sine Curve")  
plt.xlabel("x")  
plt.ylabel("sin(x)");
```



Озаглавяване на оси

Позицията, размерът и стилът на тези етикети могат да бъдат коригирани с помощта на незадължителни аргументи към функцията.

```
In [15]: plt.plot(x, np.sin(x), '-g', label='sin(x)')  
plt.plot(x, np.cos(x), ':b', label='cos(x)')  
plt.axis('equal')  
  
plt.legend();
```



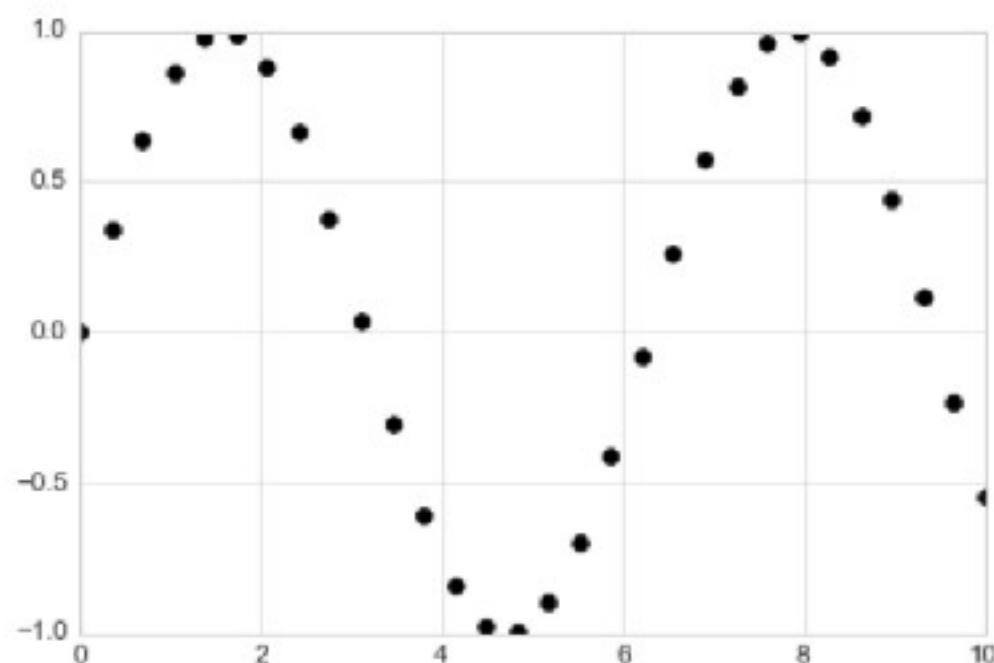
Функцията **`plt.legend()`** проследява стила и цвета на линията и ги съчетава с правилния етикет.

Simple Scatter Plots

Вместо точките да се съединяват с отсечки от линии, тук точките се представят поотделно с точка, кръг или друга форма. Вече разгледахме ***plt.plot*** / ***ax.plot***, за да създадем линейни графики. Същата тази функция може да генерира и разпръснати графики:

```
In [2]: x = np.linspace(0, 10, 30)
        y = np.sin(x)

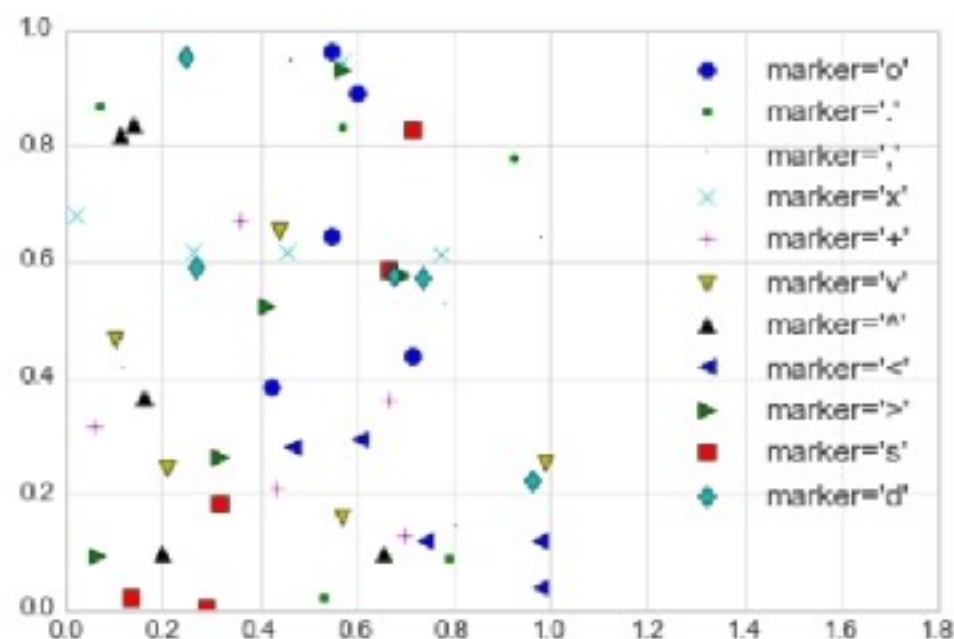
        plt.plot(x, y, 'o', color='black');
```



Simple Scatter Plots

Третият аргумент в извикването на функцията е символ, който представлява типа символ, използван за начертаване.

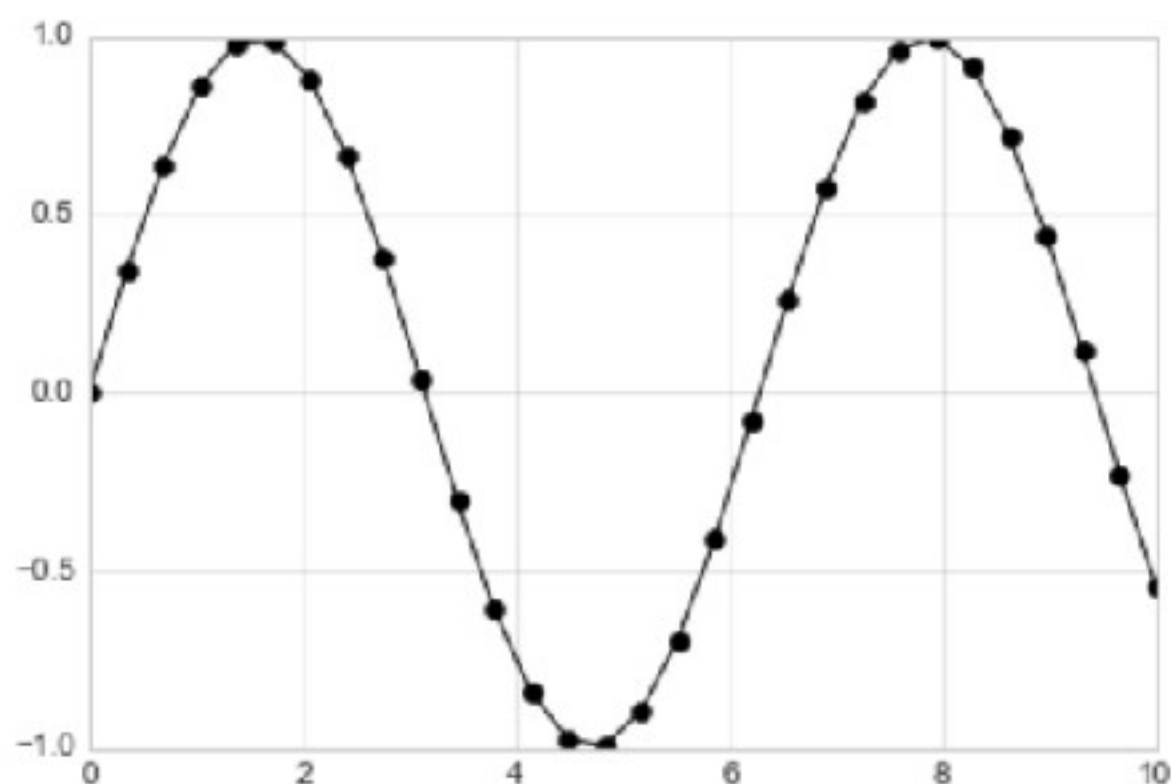
```
In [3]: rng = np.random.RandomState(0)
for marker in ['o', '.', ',', 'x', '+', 'v', '^', '<', '>', 's', 'd']:
    plt.plot(rng.rand(5), rng.rand(5), marker,
             label="marker='{0}'".format(marker))
plt.legend(numpoints=1)
plt.xlim(0, 1.8);
```



Simple Scatter Plots

Тези символни кодове могат да се използват заедно с линейни и цветни кодове за нанасяне на точки заедно с линия, която ги свързва:

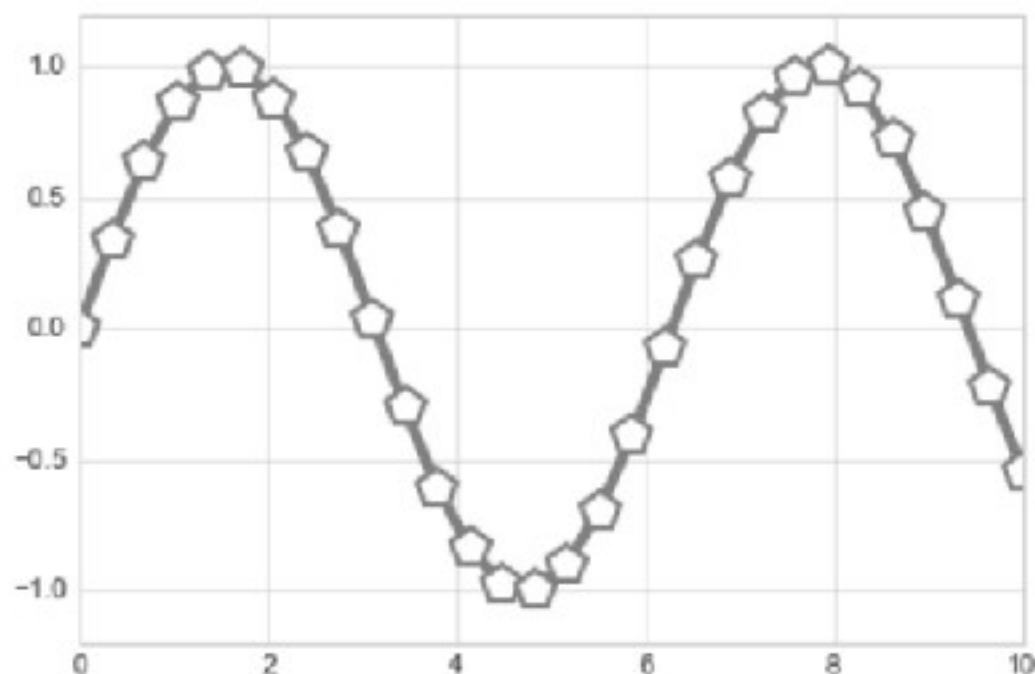
```
In [4]: plt.plot(x, y, '-ok');
```



Simple Scatter Plots

Допълнителни аргументи на ключови думи към ***plt.plot*** указват широк набор от свойства на редовете и маркерите:

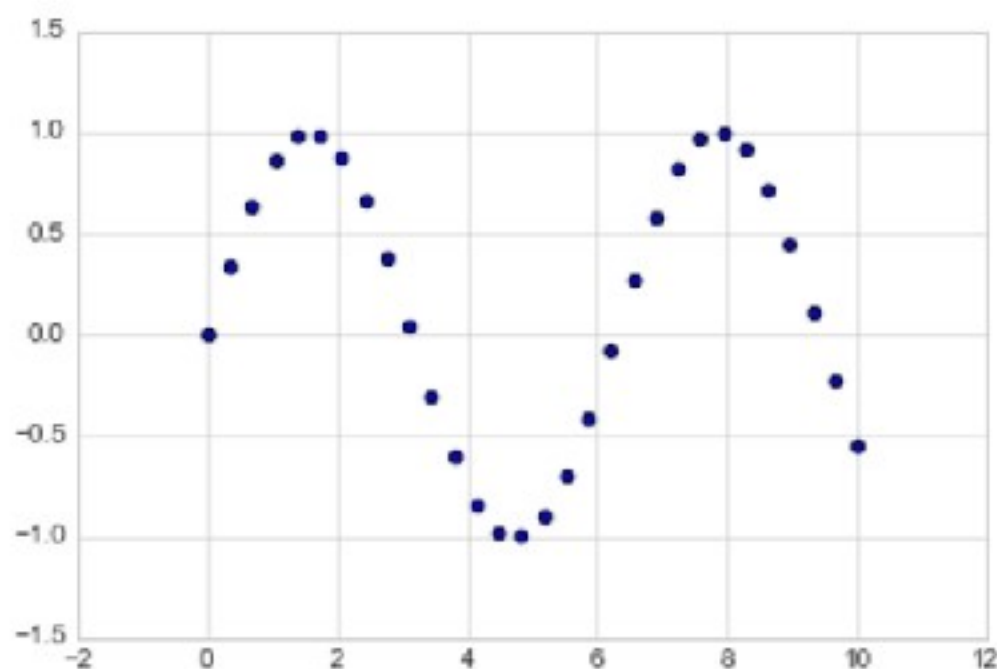
```
In [5]: plt.plot(x, y, '-p', color='gray',  
                markersize=15, linewidth=4,  
                markerfacecolor='white',  
                markeredgecolor='gray',  
                markeredgewidth=2)  
plt.ylim(-1.2, 1.2);
```



Функция `plt.scatter`

Втори метод за създаване на разпръснати графики е функцията `plt.scatter`:

```
In [6]: plt.scatter(x, y, marker='o');
```



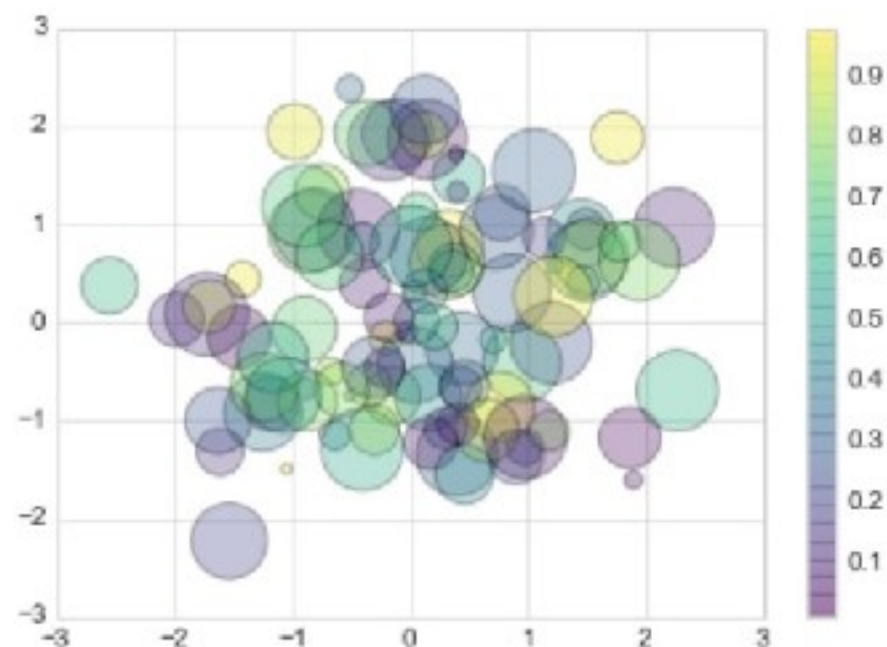
Основната разлика на **`plt.scatter`** от **`plt.plot`** е, че може да се използва за създаване на разпръснати графики, където свойствата на всяка отделна точка (размер, цвят и т.н.) могат да бъдат индивидуално контролирани или картографирани.

Функция plt.scatter

Пример: Създаване на произволен разпръснат сюжет с точки от много цветове и размери. За да видим по-добре припокриващите се резултати се използваме ключовата дума `alpha`, за да се регулира нивото на прозрачност:

```
In [7]: rng = np.random.RandomState(0)
x = rng.randn(100)
y = rng.randn(100)
colors = rng.rand(100)
sizes = 1000 * rng.rand(100)

plt.scatter(x, y, c=colors, s=sizes, alpha=0.3,
            cmap='viridis')
plt.colorbar(); # show color scale
```



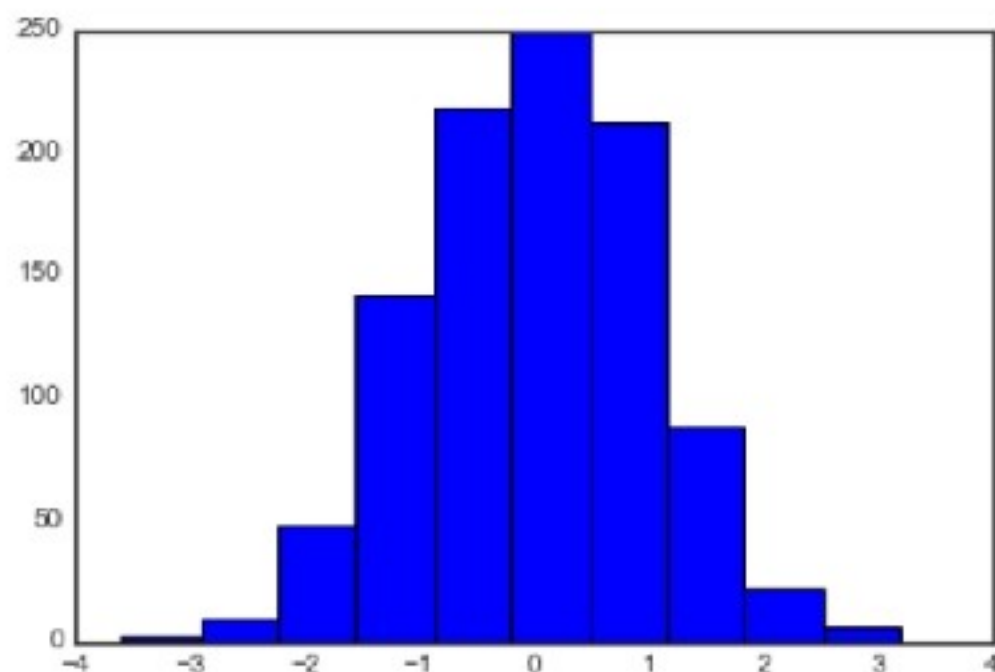
Хистограми

Създаване на проста хистограма:

```
In [1]: %matplotlib inline
import numpy as np
import matplotlib.pyplot as plt
plt.style.use('seaborn-white')

data = np.random.randn(1000)
```

```
In [2]: plt.hist(data);
```



Функцията ***hist()*** има много опции за настройка.

Персонализиране на легенди на графики

Легендите придават значение на визуализацията. Най-простата легенда може да бъде създадена с командата **`plt.legend()`**, която автоматично създава легенда за всички етикетирани елементи на графиката:

```
In [1]: import matplotlib.pyplot as plt  
plt.style.use('classic')
```

```
In [2]: %matplotlib inline  
import numpy as np
```

```
In [3]: x = np.linspace(0, 10, 1000)  
fig, ax = plt.subplots()  
ax.plot(x, np.sin(x), '-b', label='Sine')  
ax.plot(x, np.cos(x), '--r', label='Cosine')  
ax.axis('equal')  
leg = ax.legend();
```

